

CI/CD

CI 구현하기



첨부 파일 : A02.설정용 파일.zip, spring_initializer_coffee_3.5.10_java_21.zip

≡ CI 구현하기

≡ CI 구현 개요

CI 구현을 위한 로컬 프로젝트와 GitHub Repository 구조입니다.

실습 구조



스프링 프로젝트 : D:\coffee 프로젝트

깃허브 주소 : <https://github.com/seoljinuk/cicd.git>

≡ CI 구현하기

≡ 작성할 파일

실습에서 작성해야 할 파일 목록들입니다.

파일 내역

항목	설명
Image 이름	springboot-img
Container 이름	springboot-ctr
Docker File 이름	SpringDockerFile
ci 설정 파일	ci.yml
cd 설정 파일	cd.yml
Docker Compose File 이름	docker-compose.yml

≡ CI 구현하기

≡ 구현 목표

CI를 구현하기 위한 Process는 다음과 같습니다.

CI 구현 Process

원격 저장소 생성

- 저장소 주소를 클립 보드에 복사

Spring Initializer

- zip 파일 생성

Spring Project(by IntelliJ)

- 메인 컨트롤러
- application.properties
- pom.xml

Spring과 GitHub 연동

- VCS로 활성화
- 원격 저장소 지정하기
- GitHub에 Push 하기

GitHub Actions 설정

- ci.yml 파일 작성
- GitHub에 Push 하기
- Actions Tab 확인하기

CI 구현하기

원격 저장소

CI를 실습하기 위한 원격 저장소(Remote Repository)를 생성합니다.

The screenshot shows the GitHub interface for a repository named 'cicd' by user 'seoljinuk'. The repository is public. The page includes navigation tabs for Code, Issues, Pull requests, Actions, Projects, Wiki, Security, Insights, and Settings. Below the repository name, there are buttons for Pin, Watch (0), Fork (0), and Star (0). The main content area has two cards: 'Set up GitHub Copilot' and 'Add collaborators to this repository'. At the bottom, there is a 'Quick setup' section with options for 'Set up in Desktop', 'HTTPS', and 'SSH'. The 'HTTPS' option is selected, and the URL 'https://github.com/seoljinuk/cicd.git' is displayed in a text box, which is highlighted with a red border. Below the text box, there is a callout box with the text '원격 저장소 주소를 클립보드에 복사해 둡니다.' (Copy the remote repository address to the clipboard).

seoljinuk / cicd

<> Code Issues Pull requests Actions Projects Wiki Security Insights Settings

cicd Public

Pin Watch 0 Fork 0 Star 0

Set up GitHub Copilot
Use GitHub's AI pair programmer to autocomplete suggestions as you code.
Get started with GitHub Copilot

Add collaborators to this repository
Search for people using their GitHub username or email address.
Invite collaborators

Quick setup — if you've done this kind of thing before

Set up in Desktop or HTTPS SSH

https://github.com/seoljinuk/cicd.git

Get started by [creating a new file](#) or [uploading an existing file](#). [README](#), [LICENSE](#), and [.gitignore](#).

원격 저장소 주소를 클립보드에 복사해 둡니다.

Spring Initializer

Spring 프로젝트의 '기본 골격'을 만들기 위하여 Spring Initializer를 사용합니다.

참조 파일 : spring_initializer_coffee_3.5.10_java_21.zip



참조 파일을 압축 해제하고 **d:\spring_ci_cd 프로젝트**를 생성하도록 합니다.

Project

☐ Gradle - Groovy ☐ Gradle - Kotlin

☒ Maven

Language

☒ Java ☐ Kotlin ☐ Groovy

Spring Boot

☐ 4.1.0 (SNAPSHOT) ☐ 4.1.0 (M1) ☐ 4.0.3 (SNAPSHOT) ☐ 4.0.2

☐ 3.5.11 (SNAPSHOT) ☒ 3.5.10

Project Metadata

Group Artifact

Name

Description

Package name

Packaging ☒ Jar ☐ War

Configuration ☒ Properties ☐ YAML

Java ☐ 25 ☒ 21 ☐ 17

Dependencies

ADD DEPENDENCIES... CTRL + B

Spring Web ☒ WEB

Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

프로젝트 생성

Spring Initializr 사용

Project : Maven

Language : Java

Spring Boot : 3.5.10

Packaging : Jar

Configuration : Properties

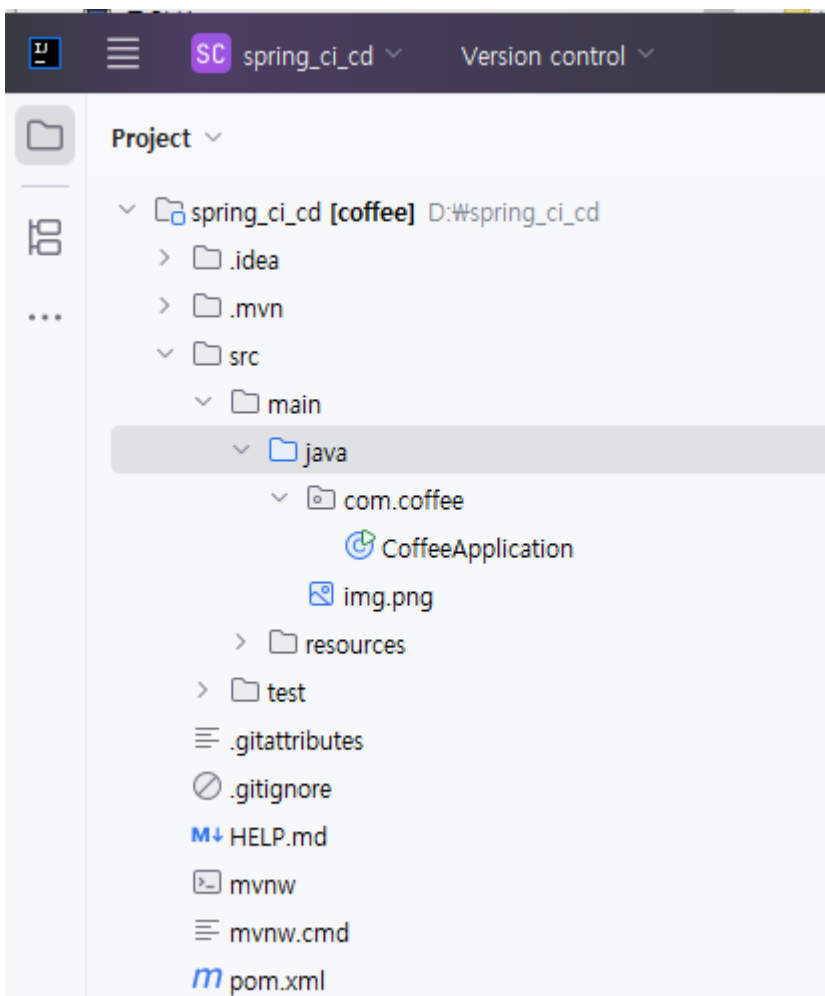
Java : 21

Dependencies : Spring Web, Thymeleaf

CI 구현하기

Spring Project

다음 화면은 스프링 프로젝트의 초기 화면입니다.



≡ Spring Project

메인 컨트롤러 MainController를 다음과 같이 작성합니다.

MainController.java

컨트롤러 작성

```
package com.coffee.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class MainController {

    @GetMapping("/")
    public String hello() {
        return "Hello GitHub Actions!";
    }
}
```

≡ CI 구현하기

≡ Spring Project

스프링 부트가 구동될 포트 번호를 지정합니다.

application.properties

포트 번호 지정

server.port=9000

Spring Project

'Web'과 '타임 리프' 및 빌드시 생성될 파일 이름을 지정합니다.

pom.xml

pom.xml

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-thymeleaf</artifactId>
  </dependency>
</dependencies>
```

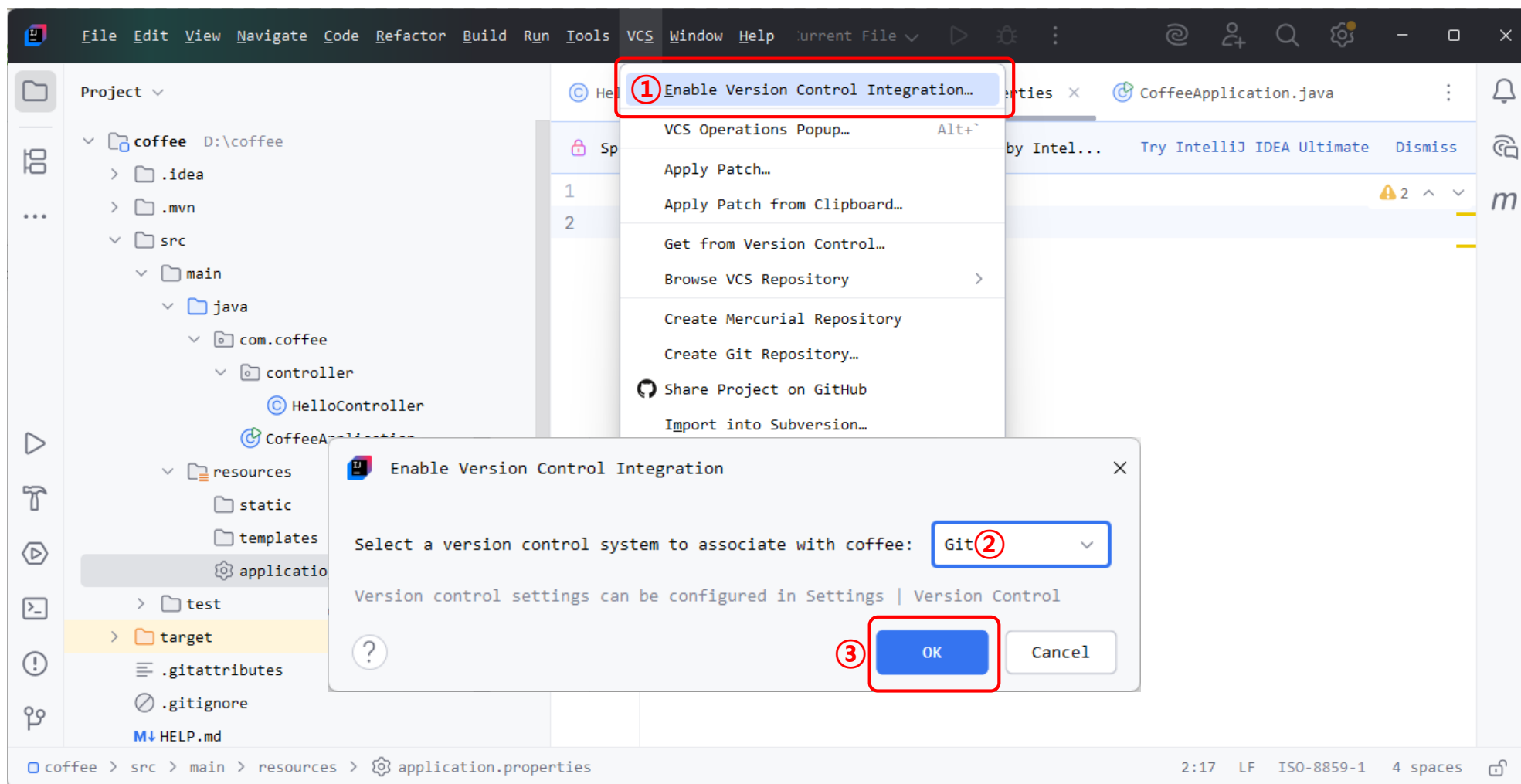
메이븐으로 jar 파일 생성시 파일 이름이 **cicd.jar**가 됩니다 .

```
<build>
  <finalName>cicd</finalName>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
```

CI 구현하기

Spring과 GitHub 연동

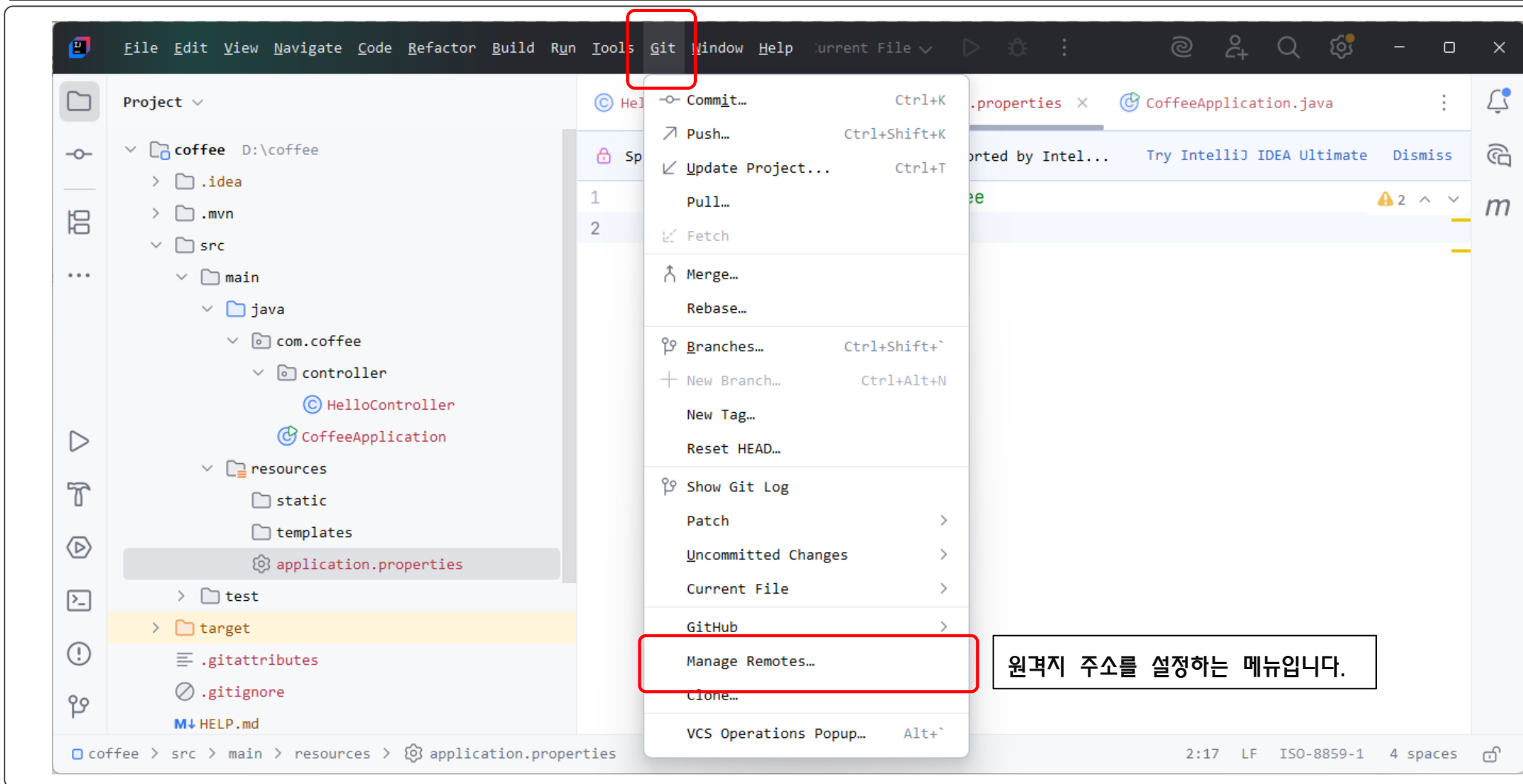
스프링 프로젝트를 로컬 저장소로 지정하기 위하여 VCS를 활성화 시키도록 하겠습니다.



CI 구현하기

Spring과 GitHub 연동

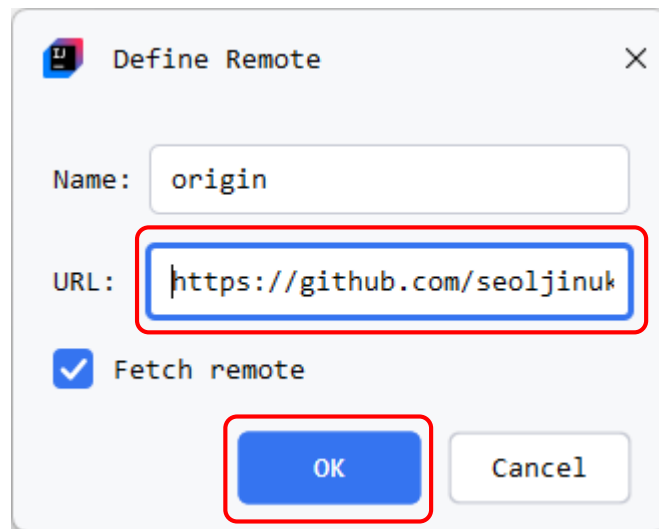
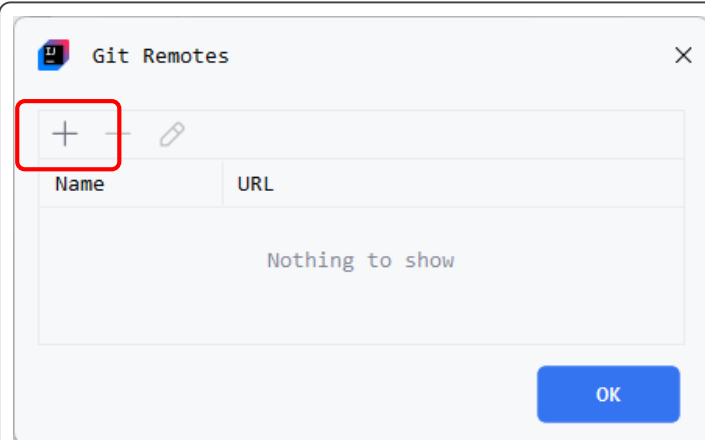
원격 저장소를 지정하기 위하여 [Manage Remotes...] 항목을 클릭합니다.



CI 구현하기

Spring과 GitHub 연동

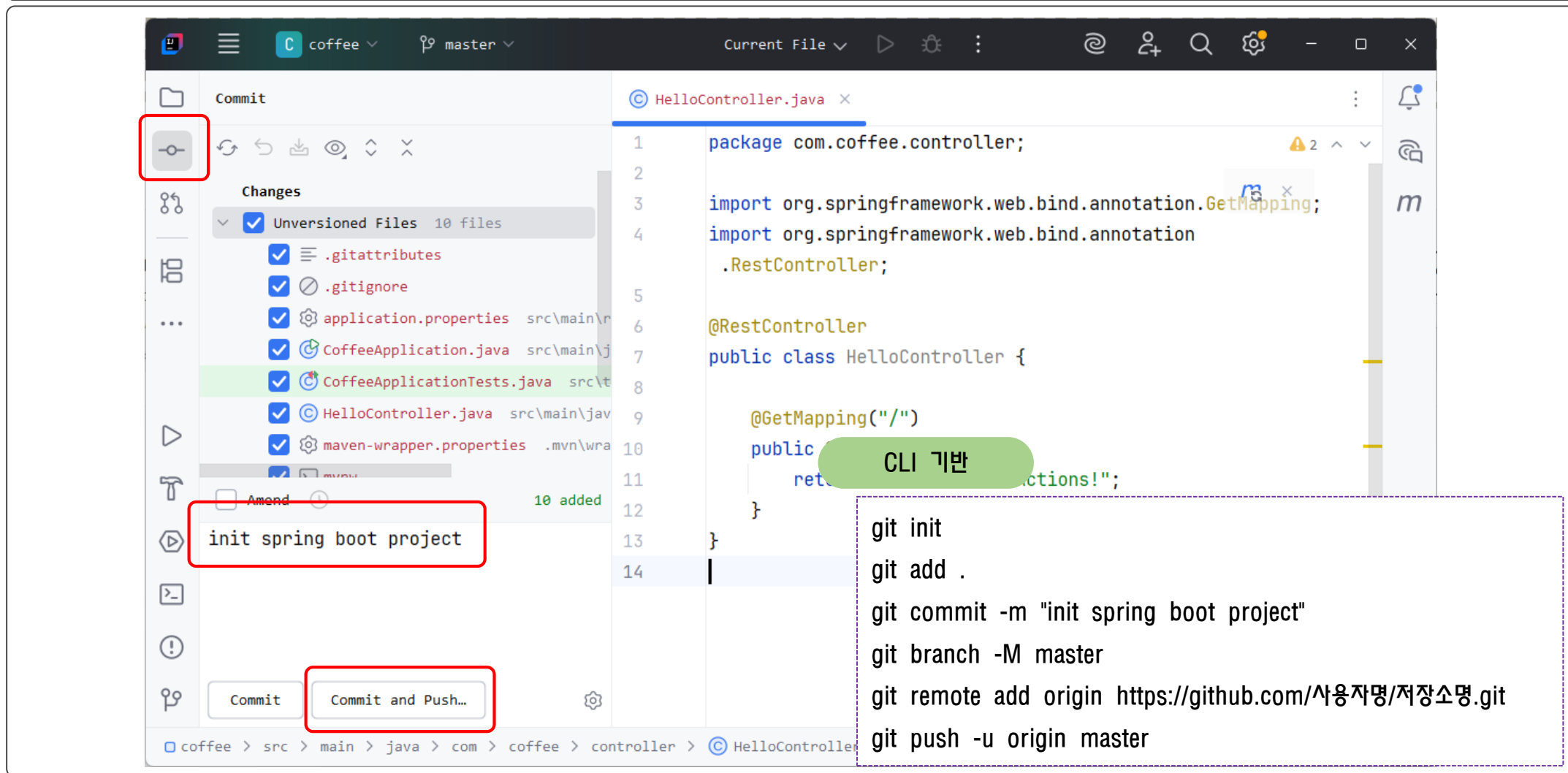
+ 기호를 클릭하여 원격 저장소 URL 주소를 등록하는 절차입니다.



☰ CI 구현하기

☰ GitHub에 Push 하기

모든 파일들을 선택하고 커밋 메시지(init spring boot project)를 입력하고, GitHub 저장소에 파일을 Push합니다.



Commit

Changes

- Unversioned Files 10 files
- ☒ .gitattributes
- ☒ .gitignore
- ☒ application.properties src/main\resources
- ☒ CoffeeApplication.java src/main\java
- ☒ CoffeeApplicationTests.java src\test\java
- ☒ HelloController.java src/main\java
- ☒ maven-wrapper.properties .mvn\wrapper

10 added

init spring boot project

Commit and Push...

CLI 기반

```
git init
git add .
git commit -m "init spring boot project"
git branch -M master
git remote add origin https://github.com/사용자명/저장소명.git
git push -u origin master
```

CI 구현하기

GitHub에 Push 하기

GitHub 원격 저장소에서 모든 파일들이 잘 Push되었는 지 브라우저에서 확인하도록 합니다.

GitHub 확인

 **cicd** Public

 Pin

 Watch 0

 master ▾

 1 Branch  0 Tags

 Go to file

t

+

 Code ▾



bluesky init spring boot project

52fe3fa · 2 minutes ago

 1 Commit



.mvn/wrapper

init spring boot project

2 minutes ago



src

init spring boot project

2 minutes ago



.gitattributes

init spring boot project

2 minutes ago



.gitignore

init spring boot project

2 minutes ago



mvnw

init spring boot project

2 minutes ago



mvnw.cmd

init spring boot project

2 minutes ago



pom.xml

init spring boot project

2 minutes ago

≡ CI 구현하기

≡ GitHub Actions 설정 하기

ci.yml 파일은 GitHub Actions에서 사용하는 CI(Continuous Integration) 설정용 파일입니다.

[ci_first.yml 파일 참고](#)

ci.yml 파일 작성

파일 작성 위치 : .github\workflows\ci.yml

```
name: Spring Boot CI
```

```
on:
```

```
  push:
```

```
    branches:
```

```
      - master
```

```
jobs:
```

```
  build:
```

```
    runs-on: ubuntu-latest
```

우측 페이지 계속 참조 바람

```
steps:
```

```
- name: Checkout source
```

```
  uses: actions/checkout@v4
```

```
- name: Set up JDK 21
```

```
  uses: actions/setup-java@v4
```

```
  with:
```

```
    java-version: '21'
```

```
    distribution: 'temurin'
```

```
    cache: maven
```

```
- name: Build with Maven
```

```
  run: mvn clean package -DskipTests
```

CI 구현하기

Push 하기

ci.yml 파일을 작성하고, 커밋 메시지(add github actions ci)를 입력한 다음 GitHub에 push하도록 합니다.

The screenshot shows an IDE window with a file named `ci.yml` open. The file content is as follows:

```
1 name: Spring Boot CI
2
3 on:
4   push:
5     branches:
6       - master
7
8 jobs:
9   build:
10    runs-on: ubuntu-latest
11
12 steps:
13   - name: Checkout source
14     uses: actions/checkout@v4
15
16   - name: Set up JDK 21
17     uses: actions/setup-java@v4
```

In the left sidebar, the 'Commit' panel is visible. It shows a list of changes with `ci.yml` selected. Below the list, the commit message 'add github actions ci' is entered and highlighted with a red box. At the bottom of the sidebar, there are buttons for 'Commit' and 'Commit and Push...'. A green callout box labeled 'CLI 기반' (CLI-based) contains the following commands:

```
git add .github/workflows/ci.yml
git commit -m "add github actions ci"
git push
```

The status bar at the bottom of the IDE shows the file path `coffee > .github > workflows > ci.yml` and the encoding `UTF-8`.

CI 구현하기

Push 내용 확인

GitHub 웹사이트에서 다음과 같이 파일/디렉토리 정보가 잘 Push 되었는 지 확인합니다.

폴더 구조

master

1 Branch

0 Tags

Go to file

Add file

<> Code

bluesky add github actions ci

8e7b511 · now

3 Commits

.github/workflows

add github actions ci

now

project

7 minutes ago

project

7 minutes ago

.gitattributes

init spring boot project

7 minutes ago

.gitignore

init spring boot project

7 minutes ago

mvnw

init spring boot project

7 minutes ago

mvnw.cmd

init spring boot project

7 minutes ago

pom.xml

init spring boot project

5 minutes ago

방금 Push한 ci.yml 파일과 연관된 폴더 목록이 보입니다.
commit message 항목도 확인합니다.

CI 구현하기

Actions Tab 확인

ci 설정의 결과물은 GitHub 저장소의 Actions Tab을 확인하면 됩니다.

The screenshot shows the GitHub Actions interface for the repository 'seoljinuk / cicd'. The 'Actions' tab is selected in the top navigation bar. On the left sidebar, under 'All workflows', the workflow 'Spring Boot CI' is highlighted with a red box. A red arrow points from this box to a text box on the right. The main area shows 'All workflows' with a search bar 'Filter workflow runs'. Below this, a table lists workflow runs. One run is shown: 'add github actions ci' on the 'master' branch, triggered by 'Spring Boot CI #1: Commit ef684a0 pushed by seoljinuk', completed 2 minutes ago in 31s. This row is also highlighted with a red box. Below the table, a text box contains instructions.

1 workflow run

Event	Status	Branch	Actor
✓ add github actions ci	Completed	master	seoljinuk

Spring Boot CI #1: Commit `ef684a0` pushed by `seoljinuk`

2 minutes ago
31s

ci.yml 파일에서 작성한 내용입니다.

"CI 구현하기"가 완료 되었습니다.

녹색 체크 박스의 링크를 클릭하여 다음 화면으로 이동합니다.

CI 구현하기

Actions Tab 확인

ci 설정의 결과물은 GitHub 저장소의 Actions Tab을 확인하면 됩니다.

Triggered via push 1 minute ago

Status

Total duration

Artifacts

 seoljinuk pushed  `e0e9684` `master`

Success

33s

—

ci.yml

on: push

 build 29s

세부 내역을 확인하기 위하여 build 녹색 링크를 클릭합니다.

CI 구현하기

Actions Tab 확인

ci 설정의 결과물은 GitHub 저장소의 Actions Tab을 확인하면 됩니다.

build

succeeded 3 minutes ago in 29s

Search logs

> ✓ Set up job	1s
> ✓ Checkout source	1s
> ✓ Set up JDK 21	0s
> ✓ Build with Maven	24s
> ✓ Post Set up JDK 21	2s
> ✓ Post Checkout source	0s
> ✓ Complete job	0s

각각의 항목을 클릭하면 상세한 History 내역을 확인할 수 있습니다.

CI/CD

가상 머신 CD 구현



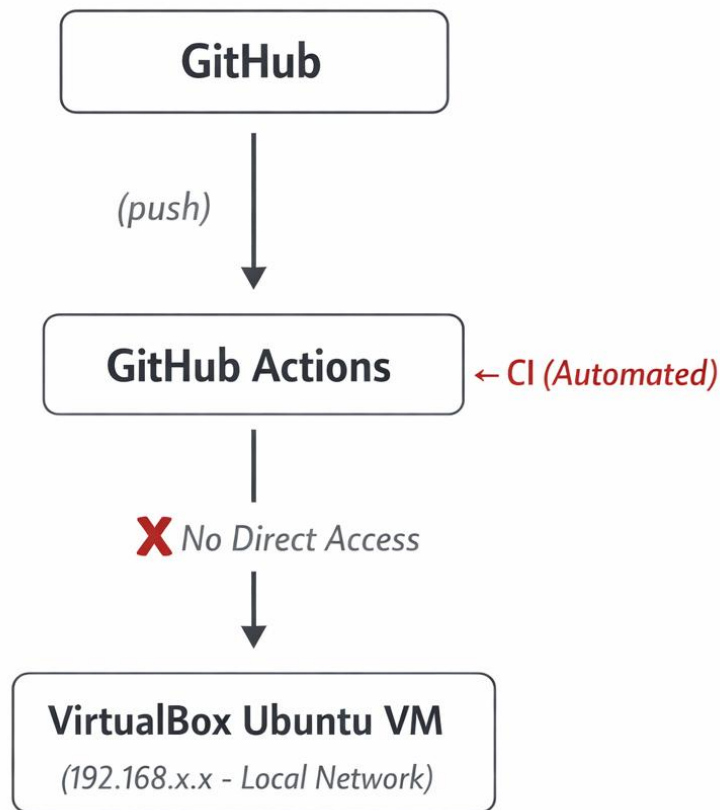
☰ 가상 머신 CD 구현

☰ CI Yes CD No

현재 상황은 서버가 클라우드가 아니고 로컬 VM 환경이므로 사람이 CD 단계를 수동으로 구현해야 합니다.

수동 CD 작업

GitHub Actions는 'GitHub 서버'에서 실행되고,
VirtualBox 안의 Ubuntu는 '내 PC 안의 로컬 서버'이기 때문에
GitHub가 마음대로 접속해서 배포(CD)할 수 없습니다.



≡ 가상 머신 CD 구현

≡ 구현 목표

로컬 컴퓨터에 가상 머신을 사용하여 CD를 구현하기 위한 Process는 다음과 같습니다.

CD 구현 Process

Spring Project(by IntelliJ)

- Docker File 작성
- Docker Compose 파일 작성
- GitHub에 Push 하기

Ubuntu 가상 머신

- 브릿지 모드
- ip 주소 고정(옵션 사항)

Build

- git clone
- jar 빌드

Docker

- 도커 설정
- 도커 이미지 생성
- 도커 컨테이너 생성(by compose)
- 컨테이너 활성화 여부 확인

브라우저에서 확인

- http://xx.xx.xx.xx
- 기본 포트로 되는 지 확인

≡ Spring Project

pom.xml 파일과 동일한 경로에 **Docker File**을 다음과 같이 **작성**하도록 합니다.

SpringDockerFile.txt 파일 참고

Dockerfile 작성

코드 내부에 명시된 **app.jar**는 Maven으로 빌드된 Spring Boot 실행 파일(JAR)을 컨테이너 안에서 쓰기 위해 붙인 '별명(이름)'이므로 **개발자가 임의의 이름을 사용해도 무방**합니다.

사용된 명령어 간단 설명

토대가 되는 이미지 정보를 지정
빌드 타임에 전달할 변수(인자)를 정의합니다.
컨테이너 이미지에 복사하는 명령어
컨테이너에서 사용할 네트워크 포트를 정의
CMD와 함께 사용해 매개 변수 전달

vi SpringDockerFile

다음과 같이 작성합니다.

```
FROM eclipse-temurin:21-jdk
ARG JAR_FILE=target/*.jar
COPY ${JAR_FILE} app.jar
EXPOSE 9000
ENTRYPOINT ["java", "-jar", "app.jar"]
```

☰ Spring Project

pom.xml 파일과 동일한 경로에 **Docker Compose**에 사용할 **설정 파일**을 작성합니다.

[docker-compose.yml 파일 참고](#)

파일) docker-compose.yml

springboot-img:latest 이라는 도커 이미지를 사용하여 springboot-ctr라는 컨테이너를 만들어 주세요.

services:

springboot:

image: springboot-img:latest

container_name: springboot-ctr

ports:

- "9000:9000"

restart: always

☰ Spring Project

Ubuntu 가상 머신에 Docker 설치를 위한 설정 파일을 작성합니다.

docker_setup.sh.txt

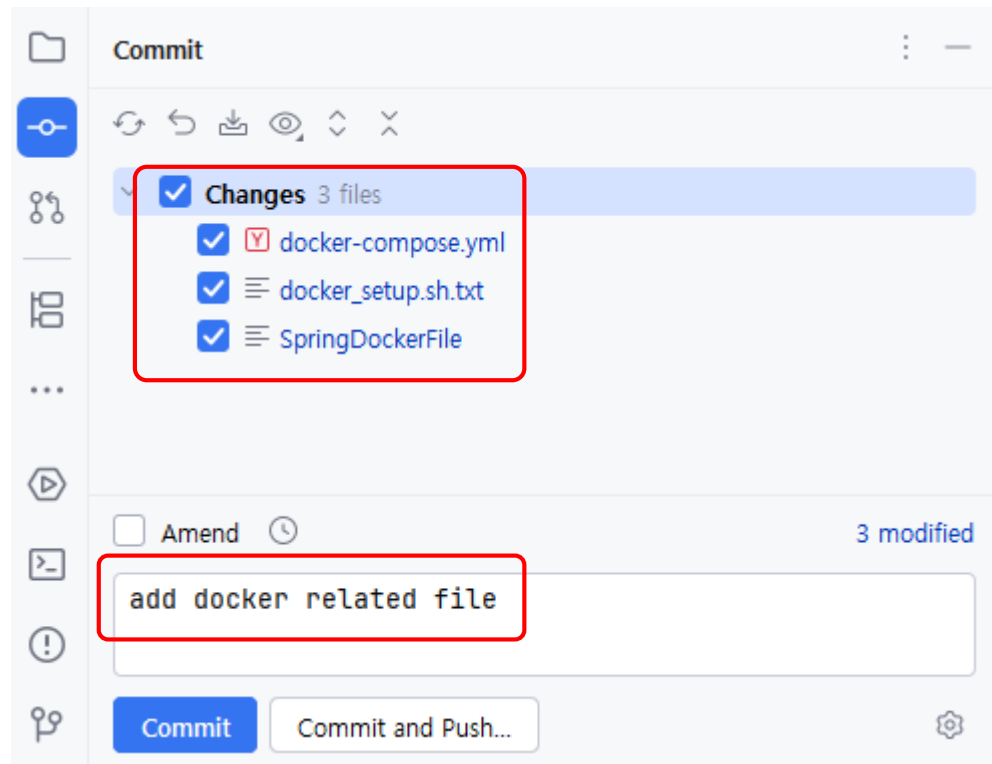
설정 파일

docker_setup.sh.txt 파일을 준비합니다.
편의상 텍스트 파일로 작성하였습니다.

가상 머신 CD 구현

Spring Project

파일 내용이 수정되었으므로 커밋 메시지(add docker related file)를 입력하고, GitHub에 Push 작업을 진행합니다.



☰ 가상 머신 CD 구현

☰ Ubuntu 가상 머신

Virtual Box에서 실습할 Ubuntu 가상 머신을 준비합니다.

특이 사항

Host Bridge 모드

ip 주소 고정하기(옵션 사항)

입력 사항

항목	입력 내용	설명
Your name	bluesky	이름이나 별칭 입력
Your Servers name	bluesky	리눅스 우분투의 이름
Pick a username	bluesky	홈 디렉토리에 사용되는 사용자 이름
Choose a password	ubuntu	사용자의 비밀 번호(ubuntu 입력)
Confirm a password	ubuntu	사용자의 비밀 번호 재입력란(ubuntu 재입력)

≡ 가상 머신 CD 구현

≡ Jar Build

Ubuntu 가상 머신에 GitHub의 소스 코드들을 Clone하고, 해당 디렉토리로 이동합니다.

클론 하기

```
cd

git clone https://github.com/seoljinuk/cicd
Cloning into 'cicd'...
remote: Enumerating objects: 35, done.
remote: Counting objects: 100%(35/35), done.
remote: Compressing objects: 100%(20/20), done.
remote: Total 35(delta 3), reused 34(delta 2), pack-reused 0(from 0)
Receiving objects: 100%(35/35), 10.20 KiB | 5.10 MiB/s, done.
Resolving deltas: 100%(3/3), done.

cd cicd
```

☰ 가상 머신 CD 구현

☰ Jar Build

Ubuntu 가상 머신에 OpenJDK를 사용하여 해당 소스 코드를 패키징합니다.

JAR 빌드

```
sudo apt update

# Java(자바)의 공식 오픈 소스 버전 OpenJDK 21
sudo apt install openjdk-21-jdk -y

# 실행 권한 부여
sudo chmod +x mvnw

./mvnw clean package -DskipTests
```

JAVA_HOME 강제 설정

```
sudo update-alternatives --config java
sudo update-alternatives --config javac
```

Docker

Ubuntu 가상 머신에 Docker를 위한 설정 작업을 합니다.

참조 파일 : docker_setup.sh.txt

Ubuntu에 Docker 설정

도커 레지스트리와 통신할 수 있는 필요한 패키지들을 설치합니다.

```
sudo cp docker_setup.sh.txt /root/docker_setup.sh
```

```
sudo su -
```

```
apt update
```

dos2unix : Windows에서 만든 쉘 스크립트를 Linux에서 실행해서 생기는 CRLF 문제 해결책

```
apt install dos2unix -y
```

```
dos2unix docker_setup.sh
```

```
chmod +x docker_setup.sh
```

```
./docker_setup.sh
```

다음 명령어를 실행하여 결과와 같이 출력이 되면 성공한 것입니다.

```
docker container --help
```

가상 머신 CD 구현

Docker

Ubuntu 가상 머신에 Docker를 위한 설정 작업을 합니다.

Ubuntu에 Docker 설정

```
systemctl status docker
```

위의 결과에서 active(running)라는 메시지가 보이면 현재 활성 상태입니다.

Ctrl + C를 눌러서 빠져 나갑니다.

다음 명령어를 사용하여 일반 사용자 모드로 이동합니다.

```
exit
```

```
sudo usermod -aG docker $USER
```

groups 명령어를 사용하여 현재 사용자(\$USER)가 docker 그룹에 속해 있는지 확인합니다.

```
groups $USER
```

bluesky : bluesky adm cdrom sudo dip plugdev lxd **docker** #
현재 사용자(bluesky)는 docker 그룹에 속해 있음을 알 수 있습니다.

도커 버전 확인 : 혹시 "permission denied"이라는 메시지를 보게 되면 logout을 수행 후 login 후 테스트 해보세요.

```
docker version
```

```
logout
```

```
docker version # 재로그인 후 도커 버전 확인
```

Client: Docker Engine - Community

Version: 29.1.3

API version: 1.52

... 중략

Server: Docker Engine - Community

Engine:

Version: 29.1.3

API version: 1.52(minimum version 1.44)

... 중략

≡ Docker

미리 생성해 둔 Docker file을 사용하여 Docker Image를 생성합니다.

Docker Image 생성

```
# 현재 디렉토리의 SpringDockerFile를 사용하여 springboot-img 이라는 Docker 이미지를 만들어 주세요.  
# 여기서 -t 옵션은 '이미지이름[:태그]' 형식으로 만들어 주는데, "태그"를 명시하지 않으면 기본 값으로 'latest'라는 단어가 붙습니다.  
# 맨 뒤의 점 주의  
cd ~/cicd
```

```
docker build -t springboot-img -f SpringDockerFile .
```

```
[+] Building 28.5s (7/7) FINISHED ... 중략
```

```
=> => transferring context: 2B 0.0s
```

```
... 중략
```

```
=> [2/2] COPY target/*.jar app.jar 5.8s
```

```
=> exporting to image 2.8s
```

```
... 중략
```

```
docker image ls
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
springboot-img:latest	a29ffe5879d0	749MB	239MB	

≡ 가상 머신 CD 구현

≡ Docker

생성된 Docker Compose 파일을 사용하여 도커 컨테이너를 생성합니다.

배포

```
# docker compose -p 생성할컨테이너이름 -f 매니페스트파일의경로와이름 up -d
```

```
docker compose -p springboot-ctr -f docker-compose.yml up -d
```

```
[+] up 2/2
```

```
✓ Network springboot-ctr_default Created 0.1s
```

```
✓ Container springboot-ctr Created 0.8s
```

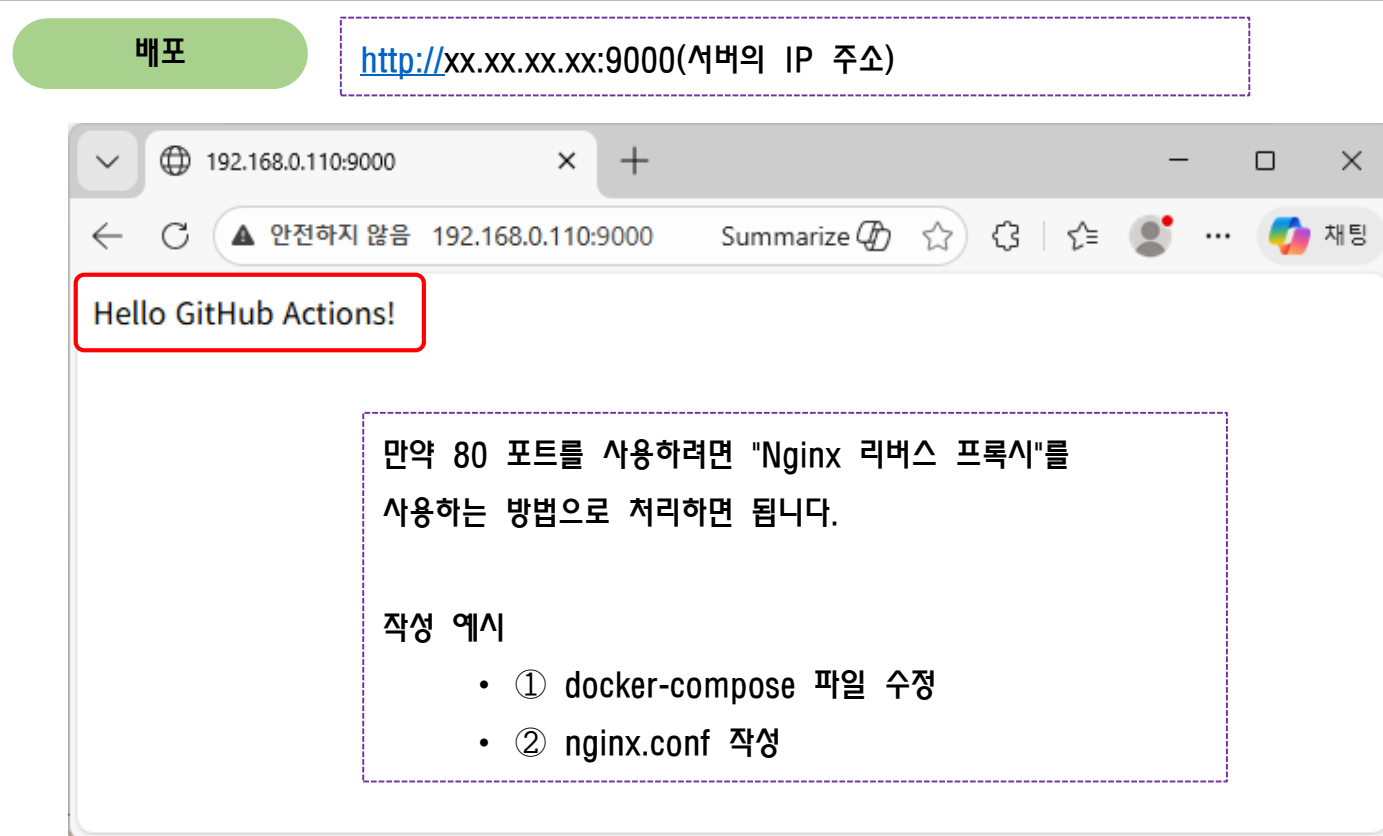
```
docker ps # 컨테이너 정보 확인 (STATUS 컬럼의 값이 Up이면 오케이)
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
479588ae7538	springboot-img:latest	"java -jar app.jar"	About a minute ago	Up About a minute
	0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp	springboot-ctr		

가상 머신 CD 구현

브라우저 확인

브라우저에서 다음과 같이 테스트를 진행합니다.



CI/CD

클라우드 CD 구현



≡ CD 구현하기

≡ 클라우드에서 CD 구현

다음은 AWS에서 CD를 할 때의 핵심 변화 포인트 목록입니다.

변동 사항

항목	Virtual Box	AWS
서버	로컬 VM	EC2
네트워크	로컬 브리지	VPC / Subnet / IGW
배포 방식	수동 SSH	GitHub Actions → EC2 자동 배포

핵심 개념

AWS를 사용하여 EC2 서버를 생성하도록 합니다.

배포(CD)는 GitHub Actions에서 담당합니다.

클라우드 CD 구현

구현 목표

클라우드 환경에 CD를 구현하기 위한 Process는 다음과 같습니다.

CD 구현 Process

Docker Hub 저장소 준비

- Repository Name, Visibility, Description
- Repository 이름 메모장에 복사

Docker Hub Access Token 생성

- Personal Access Token 생성
- Access token 정보를 메모장에 복사

GitHub Secrets

- DOCKER_USERNAME
- DOCKER_PASSWORD
- EC2_HOST
- EC2_KEY
- DOCKER_IMAGE
- DOCKER_CONTAINER

AWS 구축하기

- 키페어 생성
- Resource 생성(VPC, 보안 그룹, EC2등)
- EC2의 Public IP 메모장에 복사

MobaXTerm

- user_data.sh 스크립트 실행

Spring Project

- cd.yml 파일 작성
- GitHub에 Push하기

브라우저에서 확인

- <http://xx.xx.xx.xx> 결과 확인

Spring Project

- 컨트롤러 작성
- html 문서 작성
- image 복사
- GitHub에 Push하기

브라우저에서 확인

- <http://xx.xx.xx.xx/first> 결과 확인
- <http://xx.xx.xx.xx/second> 결과 확인

Docker Hub 저장소 <https://hub.docker.com/>

Docker Hub에서 Build 결과물인 **Docker Image**가 저장될 저장소를 준비합니다.

[Repositories](#) / [Create](#)

Create repository

Repository Name *
cicd-springboot



Short description
Spring Boot CI/CD Test

A short description to identify your repository. If the repository is public, this description is used to index your content on Docker Hub and in search engines, and is visible to users in search results.

Visibility

Using 0 of 1 private repositories. [Get more](#)

☒ **Public**
Appears in Docker Hub search results

☐ **Private**
Only visible to you

Cancel

Create

생성 예시

항목	설명
Repository Name	cicd-springboot
Visibility	Public(학습용이면 Public)
Description	Spring Boot CI/CD Test

Docker Hub 저장소

Docker Hub에서 Build 결과물이 저장될 저장소를 준비합니다.

[Repositories](#) / [cicd-springboot](#) / [General](#)

ugcadman/cicd-springboot 

Created 1 minute ago  0  0

Spring Boot CI/CD Test 

Add a category  

메모장에 기록하기
형식 : 계정아이디/Repository이름

General

Tags

Image Management

Collaborators

Webhooks

Settings

Tags  INCOMPLETE

 DOCKER SCOUT INACTIVE
[Activate](#)

Pushed images appear here.

Repository overview  INCOMPLETE

An overview describes what your image does and how to run it. It displays in [the public view of your repository](#) once you have pushed some content.

[Add overview](#)

 **buildcloud**

Build with
Docker Build Cloud

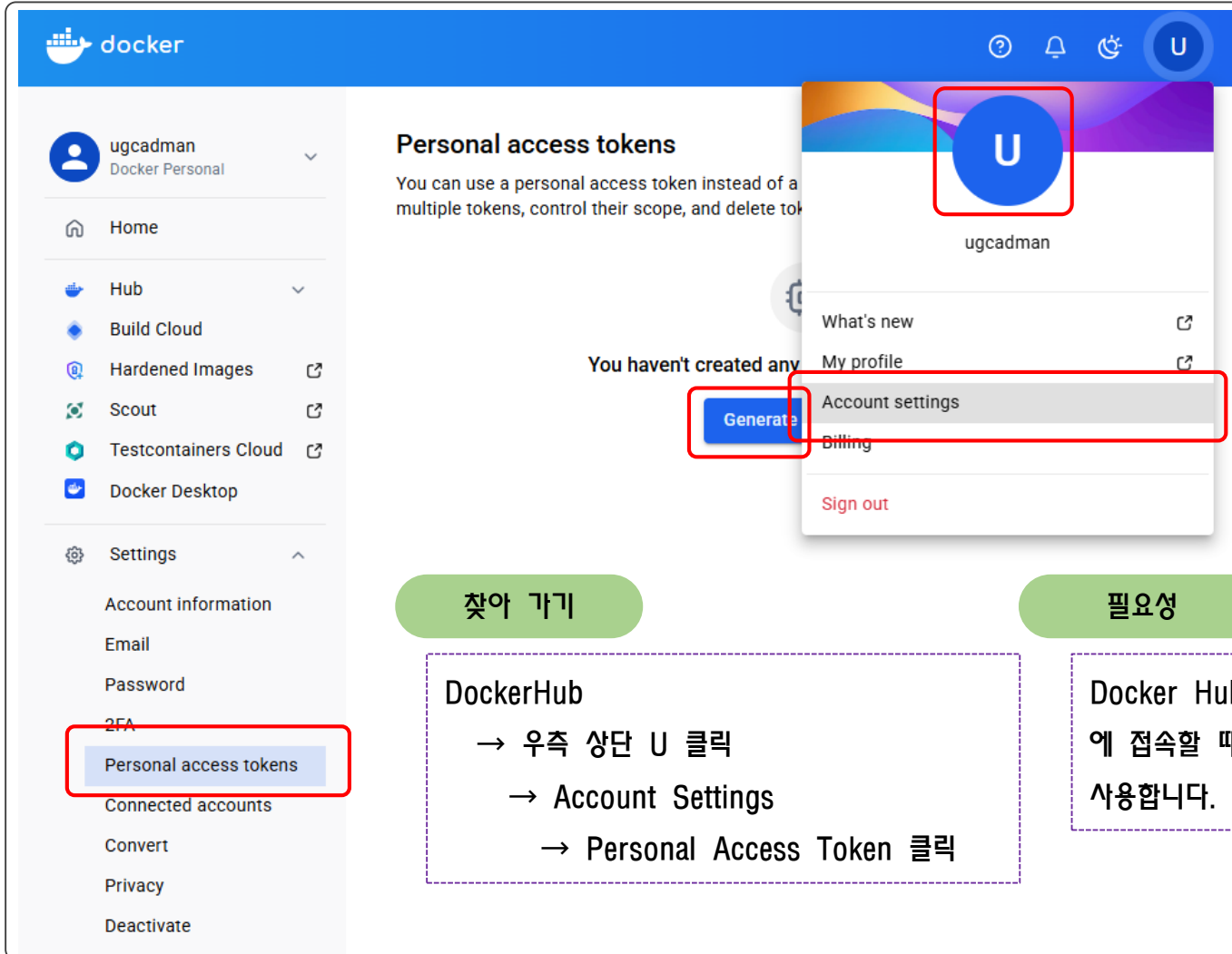
Accelerate image build times with access to cloud-based builders and shared cache.

Docker Build Cloud executes builds on optimally-dimensioned cloud infrastructure with dedicated per-organization isolation.

Get faster builds through shared caching across your team, native multi-platform support, and encrypted data transfer - all without managing

Docker Hub Access Token

DockerHub에서 Personal Access Token을 생성합니다.



Access Token은 Docker Hub에 로그인시
비밀 번호에 해당합니다.

찾아 가기

DockerHub
→ 우측 상단 U 클릭
→ Account Settings
→ Personal Access Token 클릭

필요성

Docker Hub Access Token은 CI/CD에서 나의 Docker Hub 계정에 접속할 때 "비밀번호 노출 위험"에 대한 부담을 덜어 내기 위하여 사용합니다.

Docker Hub Access Token

DockerHub에서 Personal Access Token을 생성합니다.

Create access token

A personal access token is similar to a password except you can have many tokens and revoke access to each one at any time. [Learn more](#)

Access token description

github-actions

Access token description : github-actions

Expiration date

None

Expiration date : None

Optional

Access permissions

Read & Write

Access permissions: Read / Write

Read & Write tokens allow you to p

Generate 버튼 클릭

Cancel

Generate

Docker Hub Access Token

DockerHub에서 Personal Access Token을 생성합니다.

Copy access token

Use this token as a password when you sign in from the Docker CLI client. [Learn more](#)

Make sure you copy your personal access token now. Your personal access token is only displayed once. It isn't stored and can't be retrieved later.

Access token description
github-actions

다시 추출할 수 없으니, 미리 메모장에 복사해 주세요.

Expires on
Never

Access permissions
Read & Write

To use the access token from your Docker CLI client:

1. Run

```
$ docker login -u ugcadman
```

Copy

2. At the password prompt, enter the personal access token.

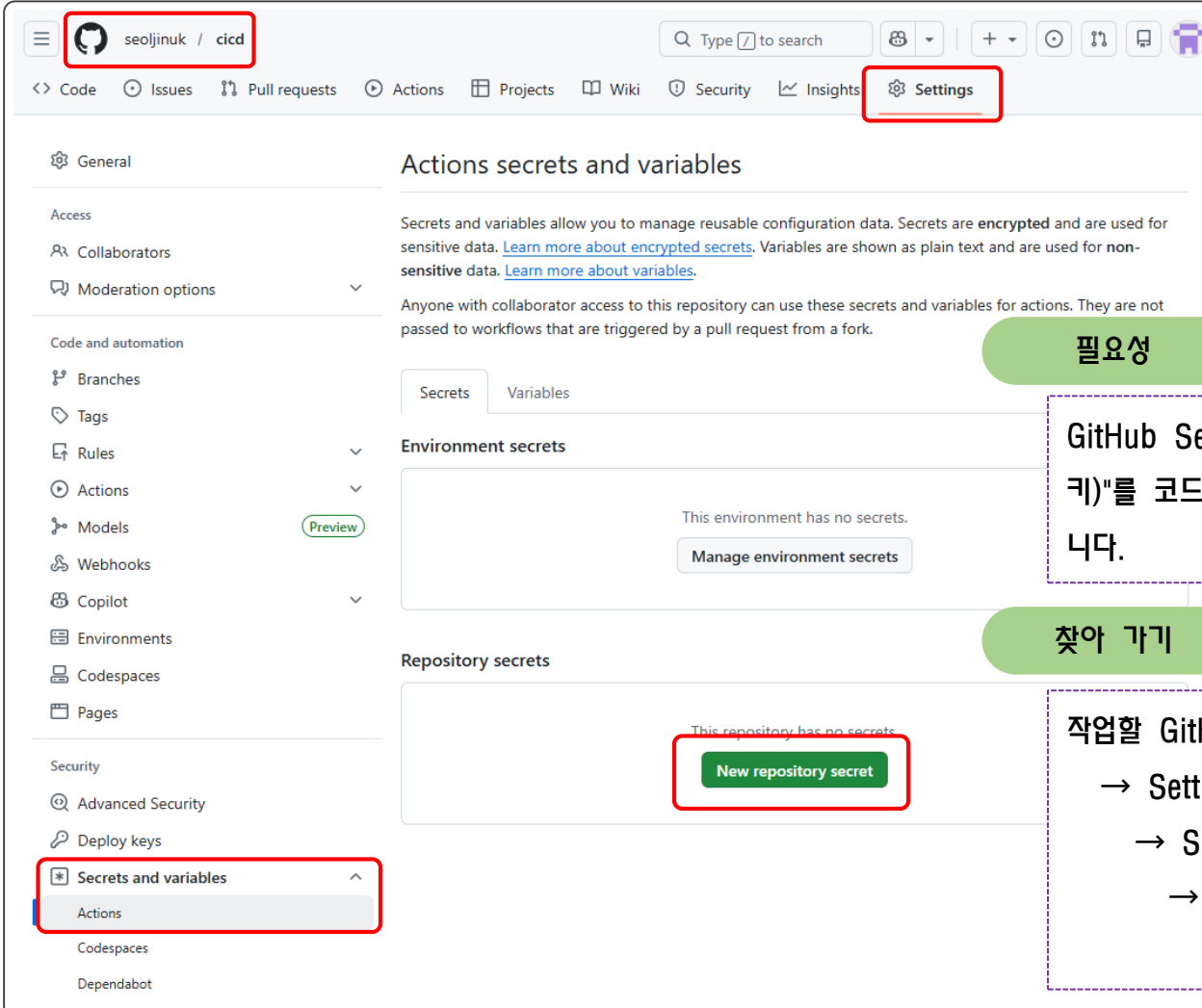
dckr_pa

Copy

Access token 정보를
메모장에 복사해 둡니다.

GitHub Secrets

CI/CD에서 필요한 "주요 비밀 정보"를 코드에 쓰지 않고, 안전하게 보관/사용하게 해주기 위한 저장소 역할을 합니다.



필요성

GitHub Secrets는 CI/CD 파이프라인에서 필요한 "비밀 정보(비밀번호, 토큰, 키)"를 코드에 쓰지 않고, 안전하게 보관 · 사용하게 해주는 저장소 역할을 합니다.

찾아 가기

작업할 GitHub Repository

→ Settings 클릭

→ Secrets and variables 클릭

→ Actions 클릭

→ New repository secret 클릭

GitHub Secrets

CI/CD에서 필요한 "주요 비밀 정보"를 코드에 쓰지 않고, **안전하게 보관/사용하게 해주기 위한 저장소** 역할을 합니다.

Secrets 설정

Name	Secret	비고
DOCKER_USERNAME	ugcadman	Docker Hub의 개인 계정
DOCKER_PASSWORD	dckr_pat_xxxxxxx	Docker Hub에서 발급 받은 Access Token
EC2_HOST	hello(차후 수정 예정)	Public IP 또는 Elastic IP
EC2_KEY	world(차후 수정 예정)	key-pair 파일(*.pem)의 내용 내용
DOCKER_IMAGE	ugcadman/springboot-img:latest	Docker Hub에 빌드할 이미지 이름
DOCKER_CONTAINER	springboot-ctr	Image로 부터 생성되는 Container 이름

설정 예시

Name *

DOCKER_USERNAME

Secret *

ugcadman

Add secret

Name 영역에 "DOCKER_USERNAME"라고 입력하고, Secret 영역에 ugcadman이라고 입력합니다.
ugcadman 대신 여러분의 Docker Hub의 개인 계정을 입력하시길 바랍니다.

"DOCKER_USERNAME"과 유사하게 필요한 항목들은 모두 Secret 항목에 추가할 수 있습니다.

GitHub Secrets

CI/CD에서 필요한 "주요 비밀 정보"를 코드에 쓰지 않고, **안전하게 보관/사용하게 해주기 위한 저장소** 역할을 합니다.

Secrets 설정

다음 그림은 GitHub Secrets에 등록된 Secret 정보입니다.

Repository secrets

[New repository secret](#)

Name ↕	Last updated
 DOCKER_CONTAINER	5 minutes ago  
 DOCKER_IMAGE	7 minutes ago  
 DOCKER_PASSWORD	8 minutes ago  
 DOCKER_USERNAME	8 minutes ago  
 EC2_HOST	now  
 EC2_KEY	now  

AWS 구축하기

다음과 같이 AWS Resource를 구축하도록 합니다.

키 페어

edu-keypair

Resource

분류	이름	설명	비고
VPC	EDU-VPC	10.250.0.0/16	65,563개 IP 사용 가능
인터넷 게이트 웨이	EDU-IGW		
Subnet	EDU-PUBLIC-SBN-2A	10.250.1.0/24	가용 영역 : 2A(마북리), 251개 IP
라우팅 테이블	EDU-PUBLIC-RT	PUBLIC 라우팅 테이블은 인터넷 게이트웨이 사용하도록 지정합니다.	
Security Group	EDU-PUBLIC-SG-2A	인바운드 규칙 80, 443, 22, 9000 Port	
EC2	EDU-PUBLIC-EC2-2A	10.250.1.240(for Private IP 주소)	
EIP	Public-2A-EIP	탄력적 IP for Public-2A(선택 사항)	

MobaXTerm

CI/CD에서 EC2를 "배포 가능한 서버"로 만들기 위한 초기 세팅을 다음과 같이 진행합니다.

참조 파일 : docker_setup.sh.txt

EC2에 Docker 설정

```
# 도커 레지스트리와 통신할 수 있는 필요한 패키지들을 설치합니다.
sudo cp docker_setup.sh.txt /root/docker_setup.sh
sudo su -
apt update

# dos2unix : Windows에서 만든 쉘 스크립트를 Linux에서 실행해서 생기는 CRLF 문제 해결책
apt install dos2unix -y
dos2unix docker_setup.sh
chmod +x docker_setup.sh
./docker_setup.sh

# 다음 명령어를 실행하여 결과와 같이 출력이 되면 성공한 것입니다.
docker container --help
```

MobaXTerm

CI/CD에서 EC2를 "배포 가능한 서버"로 만들기 위한 초기 세팅을 다음과 같이 진행합니다.

참조 파일 : docker_setup.sh.txt

EC2에 Docker 설정

```
systemctl status docker
```

위의 결과에서 active(running)라는 메시지가 보이면 현재 활성 상태입니다.

Ctrl + C를 눌러서 빠져 나갑니다.

다음 명령어를 사용하여 일반 사용자 모드로 이동합니다.

```
exit
```

```
sudo usermod -aG docker $USER
```

groups 명령어를 사용하여 현재 사용자(\$USER)가 docker 그룹에 속해 있는지 확인합니다.

```
groups $USER
```

bluesky : bluesky adm cdrom sudo dip plugdev lxd **docker** #
현재 사용자(bluesky)는 docker 그룹에 속해 있음을 알 수 있습니다.

도커 버전 확인 : 혹시 "permission denied"이라는 메시지를 보게 되면 logout을 수행 후 login 후 테스트 해보세요.

```
docker version
```

```
logout
```

```
docker version
```

 # 재로그인 후 도커 버전 확인

Client: Docker Engine - Community

Version: 29.1.3

API version: 1.52

... 중략

Server: Docker Engine - Community

Engine:

Version: 29.1.3

API version: 1.52(minimum version 1.44)

... 중략

GitHub Secrets

GitHub Secrets에서 AWS와 관련된 항목들에 대한 정보들 수정하도록 합니다.

Secrets 설정

Name	Secret	비고
EC2_HOST	xx.xx.xx.xx	Public IP 또는 Elastic IP
EC2_KEY	key-pair 파일의 내용	key-pair 파일(*.pem)의 내용 내용

EC2_HOST 항목은 AWS에서 생성한 EC2의 IP 주소 정보로 수정하도록 합니다.

EC2_KEY 항목은 key-pair 정보를 기입하도록 합니다

클라우드 CD 구현

ci.yml 파일 추가 코딩

ci.yml 파일에 도커 Hub와 관련된 코드를 추가적으로 작성하도록 합니다.

[ci_second.yml 파일 참고](#)

추가된 내용

도커 허브 리포지터리 주소는 각자의 주소를 입력해 주어야 합니다.

예시) `youraccount/springboot-img:latest`

- name: Login to DockerHub
run: `echo "${{ secrets.DOCKER_PASSWORD }}" | docker login -u "${{ secrets.DOCKER_USERNAME }}" --password-stdin`
- name: Build Docker Image
run: `docker build -t "${{ secrets.DOCKER_IMAGE }}" -f SpringDockerFile .`
- name: Push Docker Image
run: `docker push "${{ secrets.DOCKER_IMAGE }}"`

Spring Project

GitHub Actions에 추가할 CD와 관련된 파일을 작성합니다.

[cd.yml 파일 참고](#)

파일 작성

여러 페이지에 걸쳐서 작성되어 있습니다.

```
name: Spring Boot CD

on:
  workflow_run:
    workflows: ["Spring Boot CI"]
    types:
      - completed

jobs:
  deploy:
    if: ${{ github.event.workflow_run.conclusion == 'success' }}
    runs-on: ubuntu-latest
```

Spring Project

GitHub Actions에 추가할 CD와 관련된 파일을 작성합니다.

파일 작성

steps:

- name: Deploy to EC2

uses: appleboy/ssh-action@v1

with:

host: \${ secrets.EC2_HOST }

username: ubuntu

key: \${ secrets.EC2_KEY }

script: |

docker pull "\${ secrets.DOCKER_IMAGE }"

docker stop "\${ secrets.DOCKER_CONTAINER }" || true

docker rm "\${ secrets.DOCKER_CONTAINER }" || true

docker run -d --name "\${ secrets.DOCKER_CONTAINER }" -p 80:9000 "\${ secrets.DOCKER_IMAGE }"

클라우드 CD 구현

GitHub에 Push 하기

CD와 관련된 파일을 git 메시지(ci/cd file updated)와 함께 GitHub에 Push합니다.

Commit

Changes 2 files

- ☒ `cd.yml` .github\workflows
- ☒ `ci.yml` .github\workflows

Amend 1 added 1 modified

ci/cd file updated

Commit Commit and Push...

```
name: Spring Boot CD

on:
  workflow_run:
    workflows: ["Spring Boot CI"]
    types:
      - completed

jobs:
  deploy:
    if: ${ github.event.workflow_run.conclusion == 'success' }
    runs-on: ubuntu-latest

    steps:
      - name: Deploy to EC2
        uses: appleboy/ssh-action@v1
        with:
```

cicdcoffee > .github > workflows > cd.yml

클라우드 CD 구현

Actions Tab 확인

GitHub 웹사이트에서 다음과 같이 설정 정보를 확인합니다.

Actions

New workflow

All workflows

Spring Boot CD

Spring Boot CI

Management

Caches

Attestations

Runners

Usage metrics

Performance metrics

All workflows

Showing runs from all workflows

5 workflow runs

Event Status Branch Actor

Spring Boot CD

Spring Boot CD #1: completed by [seoljinuk](#)

13 minutes ago

21s

add docker related file

Spring Boot CI #5: Commit [c9708ed](#) pushed by [seoljinuk](#)

master

14 minutes ago

41s

add docker related file

Spring Boot CI #4: Commit [cf8084d](#) pushed by [seoljinuk](#)

master

Today at 2:55 PM

22s

add docker related file

Spring Boot CI #3: Commit [574a266](#) pushed by [seoljinuk](#)

master

Today at 12:59 PM

22s

add github actions ci

Spring Boot CI #2: Commit [651bf8b](#) pushed by [seoljinuk](#)

master

Today at 12:47 PM

28s

클라우드 CD 구현

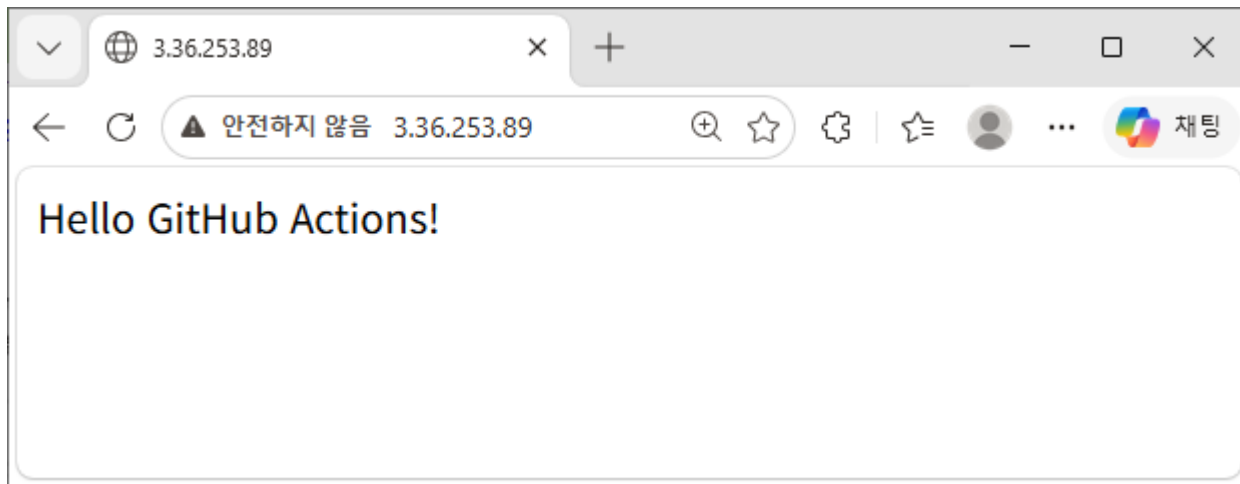
컨테이너 정보 확인

다음과 같이 MobaXTerm과 브라우저에서 EC2의 IP 주소를 입력하여 결과물이 잘 나오는 지 확인합니다.

결과 확인

```
docker container ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
03b1078a2641	ugcadman/springboot-img:latest	"java -jar app.jar"	2 minutes ago	Up 2 minutes	0.0.0.0:80->9000/tcp, [::]:80->9000/tcp	springboot-ctr



☰ Spring Project

다른 페이지로 전환하기 위하여 다음과 같이 컨트롤러를 작성합니다.

PageController.java

컨트롤러 작성

```
package com.coffee.controller;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ResponseBody;
import org.springframework.web.bind.annotation.RestController;

@Controller
public class PageController {
    @GetMapping("/first")
    public String first() { return "first"; }

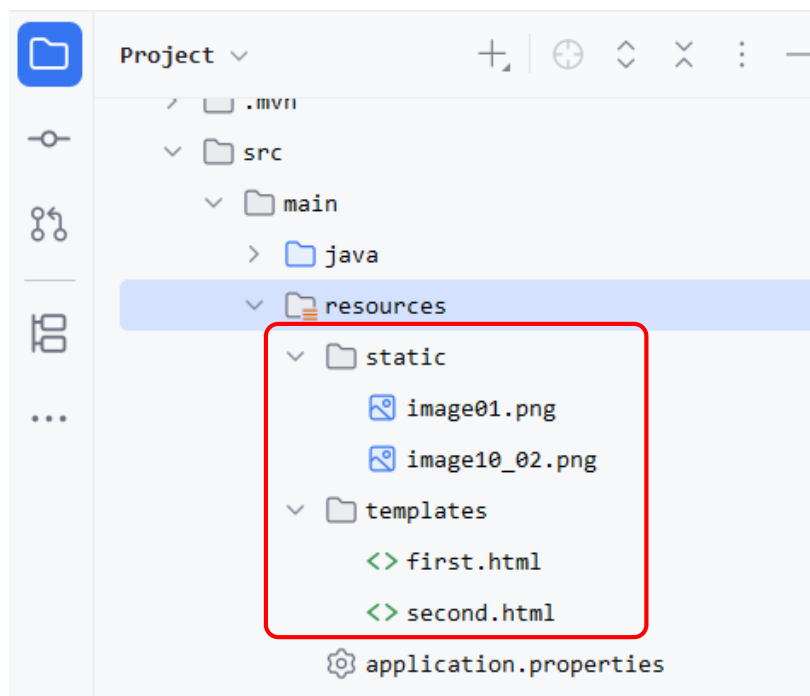
    @GetMapping("/second")
    public String second() { return "second"; }
}
```

Spring Project

결과물을 보여 줄 html 문서와 관련 이미지를 생성합니다.

관련 파일 추가

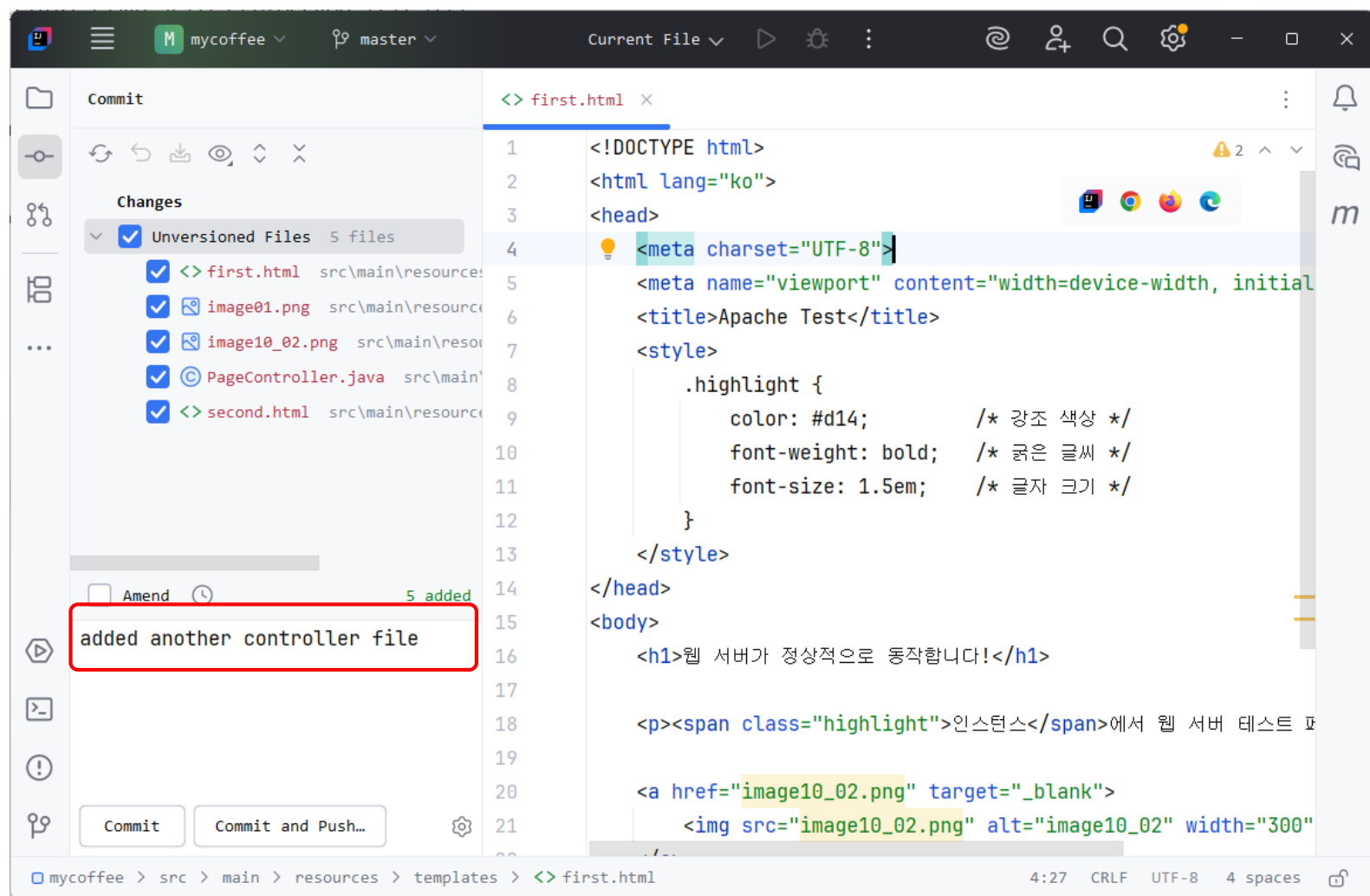
파일 현황



클라우드 CD 구현

GitHub에 Push 하기

커밋 메시지(added another controller file)와 함께 컨트롤러/html/관련 이미지들을 Push하도록 합니다.



클라우드 CD 구현

Actions Tab 확인

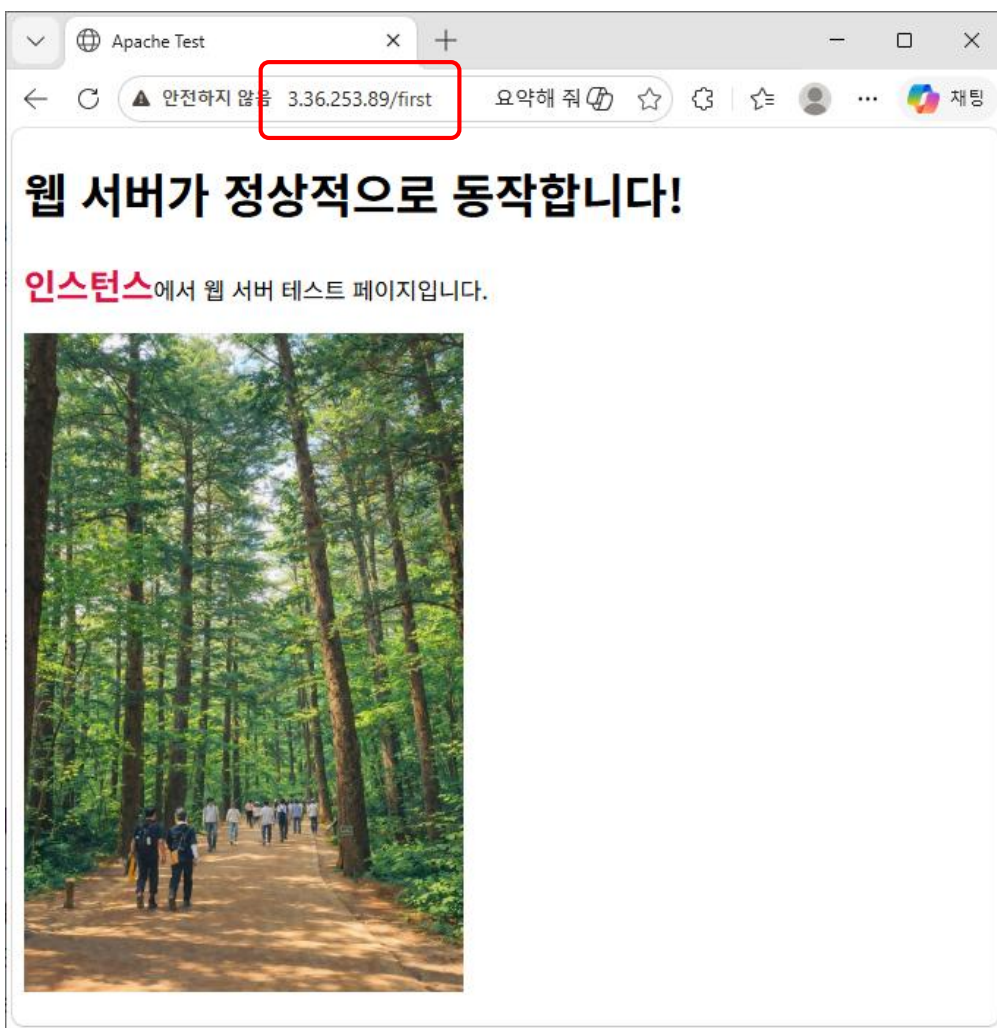
GitHub 웹사이트에서 다음과 같이 설정 정보를 확인합니다.

6 workflow runs		Event ▾	Status ▾	Branch ▾	Actor ▾
✓ Spring Boot CD	Spring Boot CD #2: completed by seoljinuk				1 minute ago 18s
✓ added another controller file	Spring Boot CI #6: Commit 940a29d pushed by seoljinuk			master	1 minute ago 40s
✓ Spring Boot CD	Spring Boot CD #1: completed by seoljinuk				23 minutes ago 21s
✓ add docker related file	Spring Boot CI #5: Commit c9708ed pushed by seoljinuk			master	24 minutes ago 41s
✓ add docker related file	Spring Boot CI #4: Commit cf8084d pushed by seoljinuk			master	Today at 2:55 PM 22s
✓ add docker related file	Spring Boot CI #3: Commit 574a266 pushed by seoljinuk			master	Today at 12:59 PM 22s
✓ add github actions ci	Spring Boot CI #2: Commit 651bf8b pushed by seoljinuk			master	Today at 12:47 PM 28s

클라우드 CD 구현

브라우저에서 확인

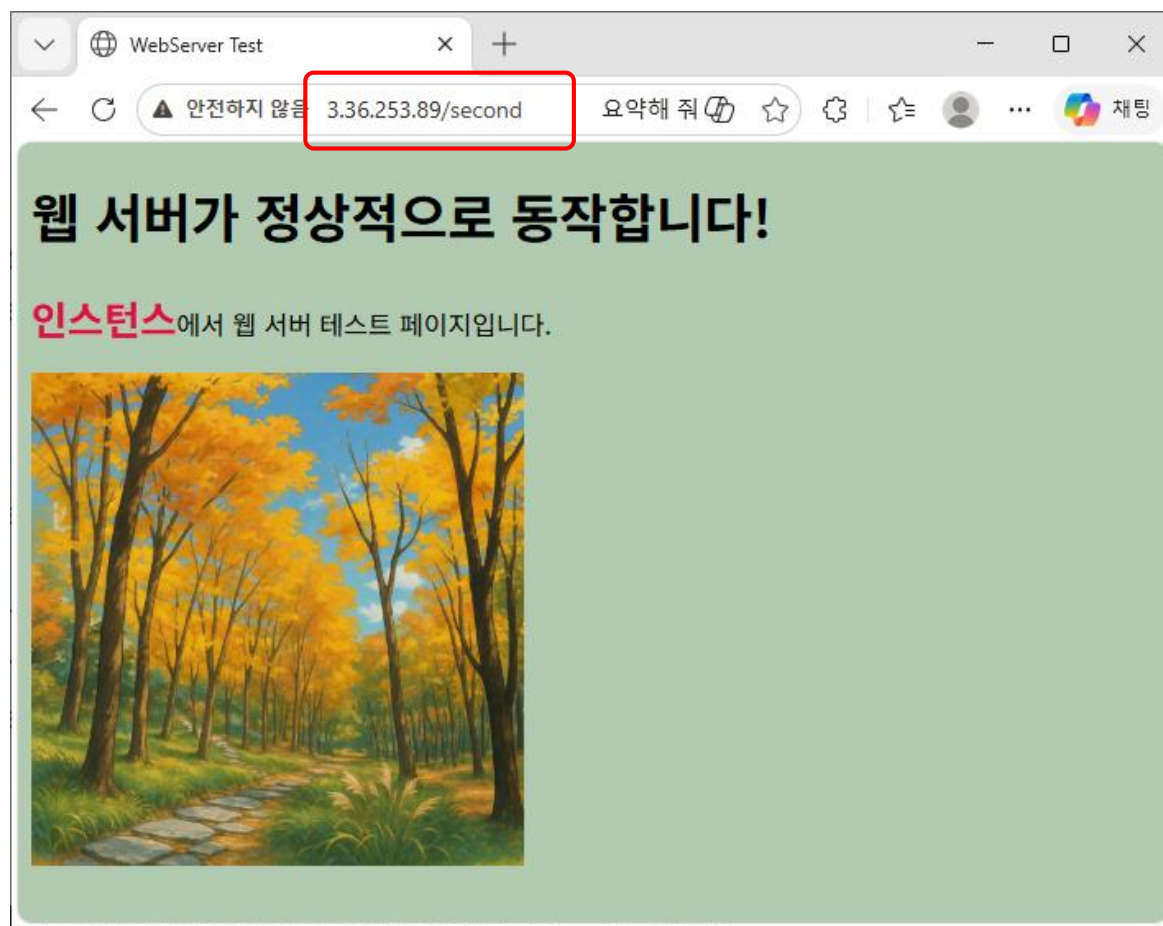
신규 컨트롤러 파일들에 대하여 잘 반영이 되는 지 확인합니다.



클라우드 CD 구현

브라우저에서 확인

신규 컨트롤러 파일들에 대하여 잘 반영이 되는 지 확인합니다.



Spring Project

다음 html 문서에서 참조하는 이미지 파일 이름을 수정하도록 합니다.

first.html

Html 파일 수정

image17_02.png 파일을 프로젝트에 추가합니다.

```
<body>
  <h1>웹 서버가 정상적으로 동작합니다!</h1>

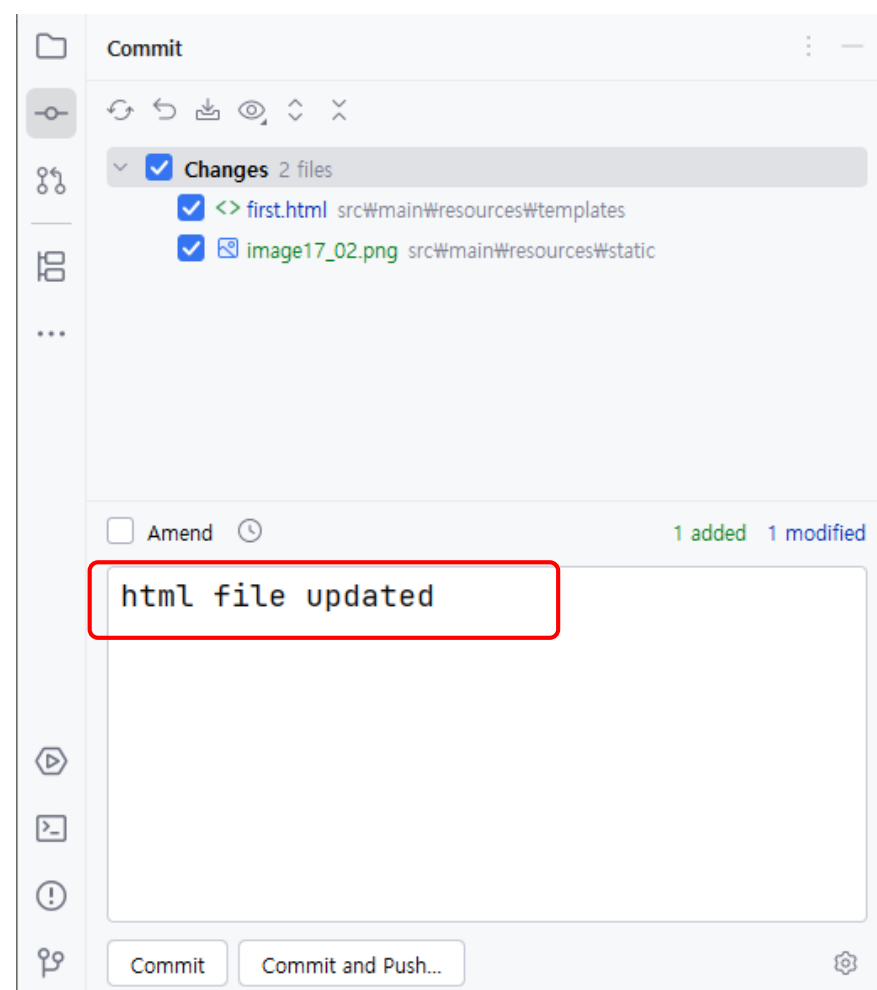
  <p><span class="highlight">인스턴스</span>에서 웹 서버 테스트 페이지입니다.</p>

  <a href="image17_02.png" target="_blank">
    
  </a>
</body>
```

클라우드 CD 구현

GitHub에 Push 하기

커밋 메시지(html file updated)와 함께 파일들을 Push하도록 합니다.



브라우저에서 확인

해당 이미지 정보가 자동으로 잘 적용이 되는 지 확인하도록 합니다.

